

Personal Project Report

Industrial Sorting Digital Twin

PLC Control of a Sorting Line
with TIA Portal, S7-PLCSIM and Factory I/O

Houénoumè Horacia Gloriéta AZONHOUMON

October 2026

This report describes a personal project conducted independently.

Repository: <https://github.com/HoraEmbedded/industrial-sorting-digital-twin>

Résumé

Ce rapport présente la conception, la programmation et la validation en simulation de la commande d'une ligne de tri automatique de caisses par hauteur. La commande est réalisée sur un automate Siemens S7-1200 avec TIA Portal, exécutée sur S7-PLCSIM et reliée à un jumeau numérique Factory I/O. Le programme gère les modes Auto et Manuel, l'arrêt en fin de cycle, l'arrêt d'urgence immédiat, le tri par rideau lumineux et plateau tournant, et le comptage par voie. L'émission anticipée des caisses fait passer la cadence mesurée de 119 à 144 caisses par heure, et les 29 tests définis sont validés. Une étude électrique de l'armoire (alimentation, départs moteurs, arrêt d'urgence matériel, câblage des entrées et sorties) complète le programme, et l'ensemble est documenté dans un dépôt GitHub. Les perspectives portent sur les indicateurs de performance embarqués, la supervision et le passage au matériel réel.

Mots-clés : automatisation industrielle, automate programmable, TIA Portal, S7-1200, jumeau numérique, Factory I/O, GRAFCET.

Abstract

This report presents the design, programming and validation in simulation of the control of an automatic box sorting line. The control runs on a Siemens S7-1200 programmed with TIA Portal, executed on S7-PLCSIM and connected to a Factory I/O digital twin. The program handles the Auto and Manual modes, a stop at the end of the cycle, an immediate emergency stop, sorting by light curtain and turntable, and counting per lane. Pipelined box emission raises the measured throughput from 119 to 144 boxes per hour, and the 29 defined tests are validated. An electrical study of the control cabinet (power supply, motor starters, hardware emergency stop, input and output wiring) complements the program, and everything is documented in a GitHub repository. The perspectives concern embedded performance indicators, supervision and the move to real hardware.

Keywords: industrial automation, PLC, TIA Portal, S7-1200, digital twin, Factory I/O, virtual commissioning, GRAFCET.

Contents

Résumé	i
Abstract	i
List of figures	iv
List of tables	v
List of abbreviations	vi
General introduction	1
1 Context and state of the art	2
1.1 PLC programming	2
1.2 Sequential control and GRAFCET	2
1.3 Digital twin and virtual commissioning	2
1.4 Machine safety	2
1.5 Positioning of the project	3
2 System specification and architecture	4
2.1 The simulated line	4
2.2 Functional requirements	5
2.3 Co-simulation architecture	6
2.4 Interface between the program and the twin	7
2.5 Summary	7
3 PLC program design	8
3.1 Program structure	8
3.2 Design principles	9
3.3 Modes and safety	9
3.4 Output management	11
3.5 Summary	12
4 Sorting sequence	13
4.1 Principle	13
4.2 GRAFCET	13
4.3 Pipelined emission	14
4.4 Counting and implementation	14
4.5 Summary	16

5	Electrical design of the control cabinet	17
5.1	Scope and assumptions	17
5.2	Cabinet architecture	17
5.3	Input and output allocation	18
5.4	24 V DC supply	18
5.5	Emergency stop and interlocks	18
5.6	Drawings	19
5.7	Summary	19
6	Validation and results	20
6.1	Test plan	20
6.2	Throughput	20
6.3	Embedded performance indicators	21
6.3.1	Interface	21
6.3.2	Computation	22
6.3.3	Comparison with the stopwatch measurement	22
6.3.4	Use on an HMI	22
6.4	Decisions taken during commissioning	23
6.5	Project organization	23
	General conclusion	25
	References	26
A	Input and output lists	27
B	Electrical drawings	29

List of Figures

2.1	Factory I/O scene <i>Sorting by Height (Advanced)</i>	4
2.2	Light curtain measuring the height of the boxes.	5
2.3	Operator panel of the scene.	5
2.4	Co-simulation architecture.	6
3.1	Data flow between the blocks of the program.	9
3.2	Fault latch, stop request and run latch in <code>FB_ModeManager</code>	11
4.1	GRAFCET of the sorting sequence.	13
4.2	Transition networks of <code>FB_SortingLogic</code>	15
4.3	Actuator command networks of <code>FB_SortingLogic</code> (two parts).	15
6.1	Watch table during a sorting cycle (two parts).	21
6.2	Box on the turntable during a sorting cycle.	21
6.3	Organization of the GitHub repository.	24

List of Tables

2.1	Functional requirements.	6
2.2	Development and simulation environment.	6
2.3	Naming conventions.	7
3.1	Blocks of the program.	8
3.2	Behaviour of the machine.	10
3.3	Latches of FB_ModeManager.	10
3.4	Lamps and counter display.	12
4.1	Actions of the steps.	13
4.2	Transitions of the sequence.	14
5.1	Devices of the cabinet.	17
5.2	Allocation of the inputs and outputs.	18
5.3	24 V DC budget.	18
5.4	Sheets of the electrical schematic.	19
6.1	Summary of the tests.	20
6.2	Throughput measured on 10 boxes.	20
6.3	Interface of FB_Kpi.	22
6.4	Issues identified and decisions taken.	23
A.1	Inputs of the scene (18).	27
A.2	Outputs of the scene (17).	28

List of abbreviations

CPU	Central Processing Unit
DB	Data Block
DI, DQ	Digital Input, Digital Output
FAT, SAT	Factory Acceptance Test, Site Acceptance Test
FB	Function Block
FC	Function
GRAFCET	Graphe Fonctionnel de Commande des Étapes et Transitions
HMI	Human-Machine Interface
I/O	Inputs and Outputs
IEC	International Electrotechnical Commission
ISO	International Organization for Standardization
KPI	Key Performance Indicator
LAD	Ladder Diagram
MPCB	Motor Protection Circuit Breaker
NC, NO	Normally Closed, Normally Open
OB	Organization Block
OPC UA	Open Platform Communications Unified Architecture
PLC	Programmable Logic Controller
PNP	Positive switching (sourcing) output type
SCADA	Supervisory Control and Data Acquisition
SM	Signal Module
TIA	Totally Integrated Automation
UDT	User-defined Data Type

General introduction

Automated sorting lines are a common building block of packaging and logistics plants. Their control logic must handle normal production, operator interventions and emergency situations safely, and it must be validated before the machine runs. Testing directly on the machine is costly and can be dangerous. A digital twin, a simulation model of the machine connected to the real control program, makes it possible to write and test the program before any hardware is available. This practice is known as virtual commissioning.

This report presents the design, programming and validation of the control of an automatic box sorting line. The controller is a Siemens S7-1200 programmed with TIA Portal in ladder logic. It runs on S7-PLCSIM and is connected to a Factory I/O digital twin of a height sorting line. An electrical study of the control cabinet completes the program.

The project has five objectives.

- Specify the behaviour of the line and the interface between the controller and the twin.
- Develop a structured PLC program with Auto and Manual modes, a stop at the end of the cycle and an immediate emergency stop.
- Implement a sorting sequence that measures the height of the boxes, routes them through a turntable and counts them per lane.
- Validate the program with a documented test plan and measure the throughput.
- Study the electrical architecture of the control cabinet and document the whole project in a versioned repository.

The work is carried out entirely in simulation: no physical machine, sensor or controller is used. The twin is the Factory I/O scene *Sorting by Height (Advanced)*, used as provided. The electrical study relies on typical component ratings.

The report follows the order of the work. Chapter 1 reviews the context and the state of the art. Chapter 2 specifies the system and its architecture. Chapter 3 describes the PLC program and chapter 4 the sorting sequence. Chapter 5 presents the electrical study of the cabinet. Chapter 6 reports the tests and the measured throughput. The general conclusion summarizes the results and proposes perspectives and improvements.

CHAPTER 1

Context and state of the art

This chapter sets the context of the project and reviews the notions it relies on: PLC programming, sequential control, digital twins and virtual commissioning, and machine safety.

1.1 PLC programming

A programmable logic controller (PLC) executes its program cyclically: it reads the inputs, runs the program and writes the outputs. The IEC 61131-3 standard [1] defines the programming languages used in industry, among them the Ladder Diagram (LAD), which remains the most common language for discrete control. Function blocks encapsulate a behaviour together with its own memory and are the basis of reusable programs [6]. Good practice includes a modular structure, symbolic names that follow a convention, and a single place where each output is written, so that two parts of the program cannot give contradictory commands.

1.2 Sequential control and GRAFCET

Machines that perform ordered operations are naturally described by steps, transitions and actions. GRAFCET, standardized in IEC 60848 [2], is a graphical language for this purpose. A GRAFCET can be read by people who do not program, it serves as a specification and as a basis for tests, and it translates directly into a step variable in ladder logic.

1.3 Digital twin and virtual commissioning

A digital twin is a virtual representation of a physical system. The term covers several levels of integration, from a simple model to a system where data flows automatically in both directions between the physical and the virtual object [7, 8]. In manufacturing, a common use is virtual commissioning: the control program is connected to a simulation of the machine and tested before installation, which shortens commissioning and reduces risks [9]. In this project the twin is a Factory I/O scene [11] connected to a virtual PLC, and it plays the role of the machine. The same program logic could then be downloaded to a physical controller, with the I/O addresses adapted.

1.4 Machine safety

Safety requirements shape the control program. The emergency stop function must be available at all times and in every operating mode, it must stop hazardous movements as quickly as possible, and releasing the device must not restart the machine by itself. These principles are set by IEC 60204-1 [3] and ISO 13850 [4]. ISO 13849-1 [5] defines the performance levels of safety-related control functions. In this project the emergency stop acts in the program and, in the electrical study, through a relay that cuts the actuator supply. A certified safety relay would be required on a real machine.

1.5 Positioning of the project

The project combines the four ideas above: a structured IEC 61131-3 program, a GRAFCET-based sequence, virtual commissioning with a documented test plan, and an electrical study of the cabinet. Three aspects characterize the work. First, the interface between the program and the twin follows naming conventions that keep both sides synchronized by name. Second, every behaviour is verified by a numbered test traced to a requirement. Third, the whole project is versioned in a GitHub repository with automatic consistency checks.

CHAPTER 2

System specification and architecture

This chapter specifies what the system must do and describes how the controller and the digital twin are connected.

2.1 The simulated line

The digital twin is the Factory I/O scene *Sorting by Height (Advanced)*. Figure 2.1 shows its T-shaped layout: the entry conveyor joins the turntable at the center of the T, and the left and right exit conveyors extend on each side. Boxes of two heights are emitted at the start of the entry conveyor. They cross a light curtain that measures their height (Figure 2.2) and reach the turntable. The turntable rotates by 90 degrees and its rollers push the box onto the left or the right exit conveyor. A sensor at the end of each exit conveyor detects the box, and a remover deletes it from the scene.

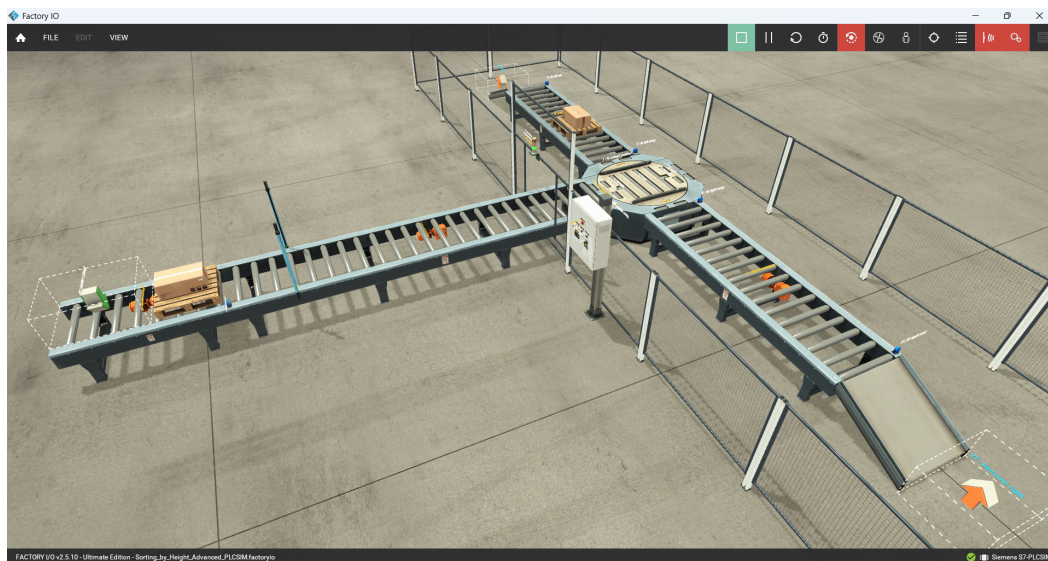


Figure 2.1. Factory I/O scene *Sorting by Height (Advanced)*.



Figure 2.2. Light curtain measuring the height of the boxes.

The operator panel (Figure 2.3) groups the Start, Stop and Reset pushbuttons with their lamps, the emergency stop, the Manual/Auto selector and a counter display. A tower light with green, yellow and red lamps signals the state of the machine.



Figure 2.3. Operator panel of the scene.

2.2 Functional requirements

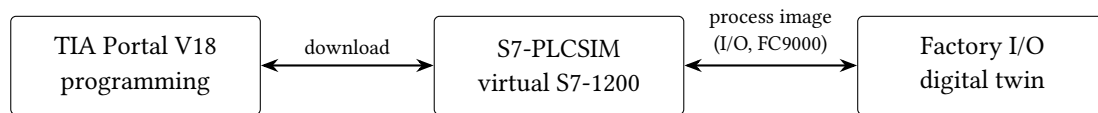
The behaviour expected from the controller is summarized in Table 2.1. Each requirement is traced to the tests of chapter 6.

Table 2.1. Functional requirements.

ID	Requirement
R1	The operator selects Auto or Manual. In Manual the machine does not run by itself and the actuators can be commanded for commissioning.
R2	In Auto mode, Start launches production if no fault is present.
R3	Stop requests a stop at the end of the cycle: no new box is emitted, the box in progress is finished and counted.
R4	The emergency stop switches off all actuators immediately and latches a fault. Reset, then Start, is required to run again.
R5	Boxes are sorted by height: low boxes to the left exit conveyor, high boxes to the right one.
R6	Boxes are counted per lane at the end of each exit conveyor, and the total is shown on the operator panel. Reset in Manual mode zeroes the counters.
R7	Lamps indicate the state: green when running, red when stopped, yellow on a fault or in Manual mode, plus the lamps of the three pushbuttons.
R8	The next box is emitted as soon as possible, without collision on the turntable.

2.3 Co-simulation architecture

The program is developed in TIA Portal V18 and downloaded to S7-PLCSIM, a virtual S7-1200. Factory I/O is configured with the S7-PLCSIM driver and exchanges the process image with the virtual PLC (Figure 2.4). Table 2.2 lists the environment.

**Figure 2.4.** Co-simulation architecture.**Table 2.2.** Development and simulation environment.

Element	Version or model	Role
TIA Portal	V18	Programming in ladder logic (LAD)
PLC	S7-1200, CPU 1214C DC/DC/DC	Controller, article 6ES7 214-1AG40-0XB0, firmware V4.6
S7-PLCSIM	Same generation as TIA Portal	Virtual PLC
Factory I/O	v2.5.10, Ultimate Edition	Digital twin of the line
Git and GitHub	Conventional Commits	Version control and documentation

Two provisions are needed for the link to work:

- the function FC9000, supplied with the Factory I/O template for S7-PLCSIM, is called first in OB1 and ensures the communication with the simulator;
- the on-board I/O of the CPU is moved to start address 100. Factory I/O writes the inputs directly in the process image, and if a physical input had the same address, the CPU would overwrite the value at every cycle.

2.4 Interface between the program and the twin

The scene exposes 18 inputs (pushbuttons, selector, position sensors, light curtain) and 17 outputs (conveyors, turntable, removers, lamps, counter display). The complete lists are given in Appendix A. The tags of the PLC mirror the names of the scene, so that the program and the twin stay synchronized by name (Table 2.3).

Table 2.3. Naming conventions.

Element	Convention	Example
Input tag	I_ followed by the scene name in PascalCase	I_AtTurntableEntry
Output tag	Q_ followed by the scene name in PascalCase	Q_EntryConveyor
Block	FB_, FC_, DB_ or UDT_ followed by a PascalCase name	FB_ModeManager
Block parameter	i_ input, o_ output, io_ in/out, s_ static, t_ temporary	i_StopNC

The polarity of the signals follows industrial practice: Stop and emergency stop are normally closed (TRUE at rest, FALSE when pressed or broken), Start and Reset are normally open.

2.5 Summary

The line sorts boxes of two heights with a light curtain and a turntable. Eight requirements define the expected behaviour. The controller and the twin are linked through S7-PLCSIM, with a link function called first in OB1 and an I/O addressing that avoids overlaps. A naming convention keeps both sides consistent.

CHAPTER 3

PLC program design

This chapter describes the structure of the PLC program, the management of the operating modes and of the safety functions, and the writing of the outputs. The sorting sequence itself is detailed in chapter 4.

3.1 Program structure

The program is made of six code blocks and two data structures (Table 3.1). OB1 calls the blocks in a fixed order, shown in Figure 3.1: the link function FC9000, the mode manager, a small network that requests the clearing of the counters when Reset is pressed in Manual mode, the sorting logic, and finally the output function.

Table 3.1. Blocks of the program.

Block	Role
OB1 Main	Calls the blocks below in order
FC9000	Link with S7-PLCSIM supplied with the Factory I/O template, always called first
FB_ModeManager	Auto and Manual modes, Start, stop request, emergency stop fault latch
FB_SortingLogic	Sorting sequence, box emission, counting
FC_Outputs	Single writer of all outputs, with safety gating, lamps and counter display
DB_Machine	Machine state, actuator commands for Auto and Manual, counters
UDT_ActuatorCmd	One command bit per actuator
FB_Kpi	Availability, throughput and average cycle time computed in the PLC from run-time and Auto-mode counters

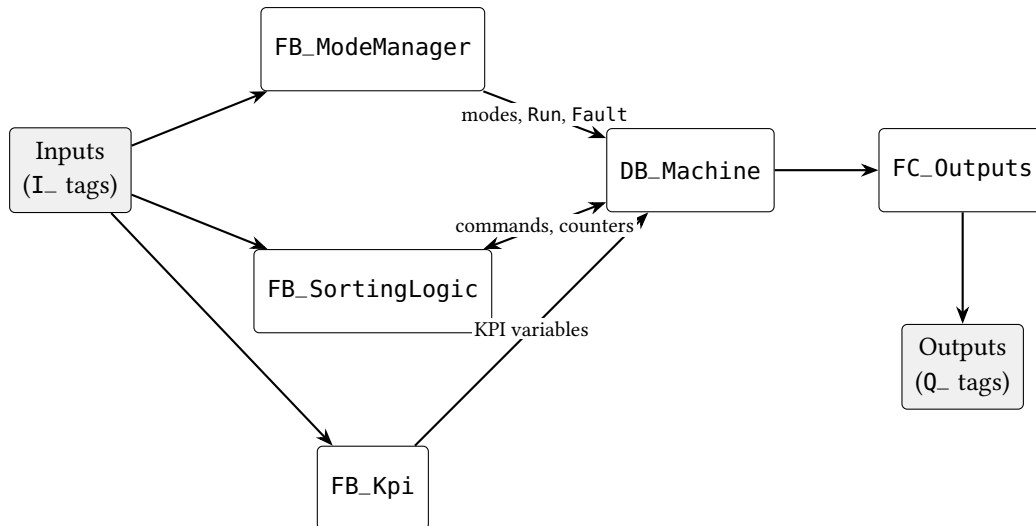


Figure 3.1. Data flow between the blocks of the program.

Figure 3.1 shows that the inputs feed the mode manager and the sorting logic. Both exchange data with DB_Machine, which FC_Outputs reads before writing all the outputs. FC_Outputs also reads the emergency stop input directly.

3.2 Design principles

- **Single writer.** Every output is written in exactly one place, FC_Outputs, which removes any conflict between two parts of the program.
- **No memory bits.** The state of the program lives in DB_Machine and in the static variables of the blocks, so that a block is self-contained and reusable.
- **Parameterized blocks.** Blocks communicate through a typed interface (inputs, outputs, in/out, static), not through global tags.
- **Readable networks.** Each network has a title and a comment in English, and each block carries its author.

3.3 Modes and safety

The behaviour of the machine for the main events is given in Table 3.2. The emergency stop acts immediately and whatever the mode, and the machine never restarts by itself after its release.

Table 3.2. Behaviour of the machine.

Event	Reaction
Start (Auto mode, no fault)	The machine runs
Stop	Stop at the end of the cycle: no new box is emitted, the box in progress is finished and counted, then the machine stops
Emergency stop	Immediate switch-off of all actuator outputs, fault latched
Emergency stop released	The fault stays latched until Reset is pressed, and Start is then required to run again
Change to Manual mode	The machine stops, and the actuators follow manual commands for commissioning

FB_ModeManager implements this behaviour with three latches, summarized in Table 3.3. Figure 3.2 shows them in ladder logic.

Table 3.3. Latches of FB_ModeManager.

Latch	Set	Reset
Fault	Emergency stop pressed	Reset pressed while the emergency stop is released
Stop request	Stop pressed while running	The machine is no longer running
Run	Start pressed in Auto mode, with the emergency stop and Stop healthy and no fault	Fault, mode other than Auto, emergency stop pressed, or stop requested with no cycle in progress

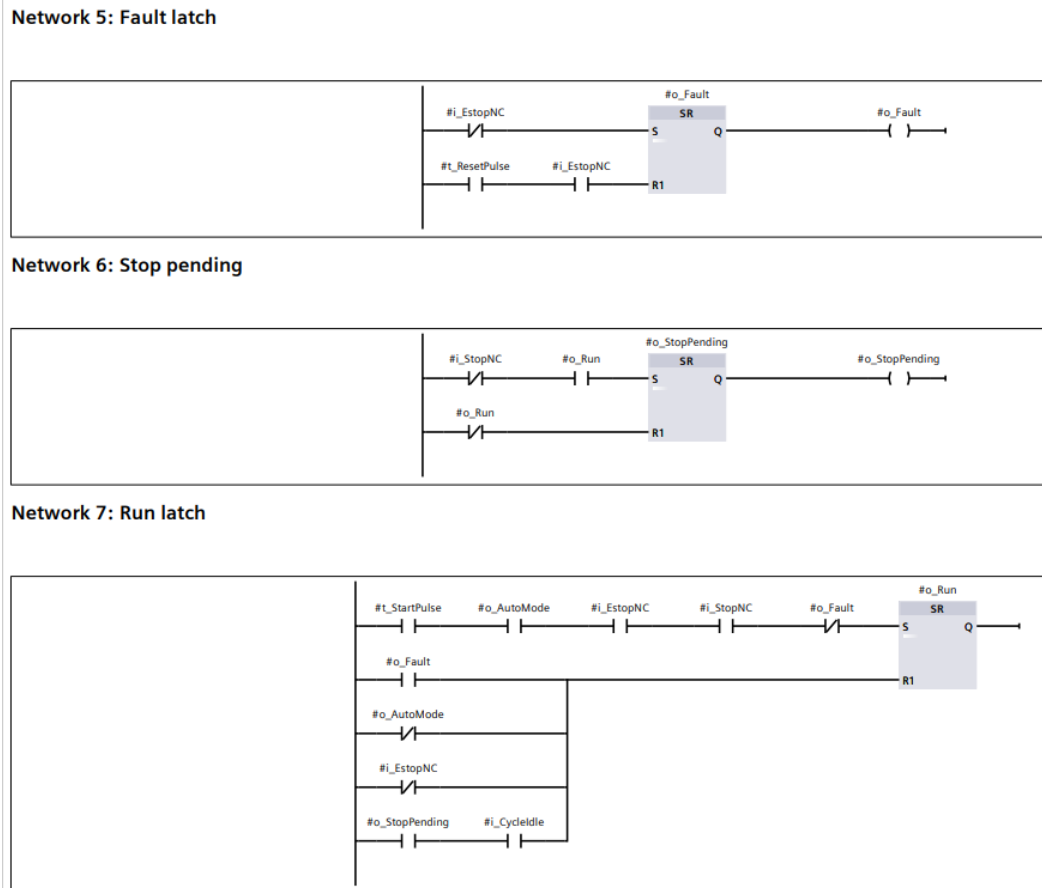


Figure 3.2. Fault latch, stop request and run latch in FB_ModeManager.

Each latch is an SR box. The fault latch is set by the emergency stop contact, which is TRUE when healthy, and it is cleared by a rising edge on Reset. The run latch combines all the conditions of Table 3.3, so that a fault or a change of mode drops the machine at once.

3.4 Output management

All outputs are written by FC_Outputs. A common enabling condition is computed first, then each actuator X is commanded from the Auto commands written by the sorting logic or from the manual commands written during commissioning:

$$\begin{aligned} \text{Enable} &= \text{I_EmergencyStop} \text{ AND NOT } \text{Fault} \\ \text{Q_X} &= \text{Enable} \text{ AND } ((\text{Run} \text{ AND } \text{AutoCmd.X}) \text{ OR } (\text{ManualMode} \text{ AND } \text{ManCmd.X})) \end{aligned}$$

The emergency stop therefore acts on every actuator in the same scan, whatever the mode. The lamps and the counter display are driven as in Table 3.4.

Table 3.4. Lamps and counter display.

Output	Condition
Green lamp and Start lamp	Run
Red lamp and Stop lamp	NOT Run
Yellow lamp	Fault OR ManualMode
Reset lamp	Fault AND emergency stop released
Counter display	CountTotal

3.5 Summary

The program is organized around a global data block, a mode manager, a sorting logic and a single output function. The emergency stop, the stop at the end of the cycle and the restart procedure are implemented with three latches and verified by tests. All outputs are written in one place.

Sorting sequence

This chapter describes the sorting sequence: how the height of a box is measured, how the sequence is organized, how the next box is emitted early, and how boxes are counted.

4.1 Principle

The light curtain provides two signals (Figure 2.2). A low box triggers only `I_LowBox`. A high box triggers `I_LowBox` and `I_HighBox`. The program therefore only needs `I_HighBox`: it latches this signal while the box travels from the emitter to the turntable (latch `s_SeenHigh`). A box that never triggers `I_HighBox` is a low box. Low boxes are sent to the left exit conveyor and high boxes to the right one.

4.2 GRAFCET

The sequence is described by the GRAFCET of Figure 4.1. It has six steps, and the transitions `t1` to `t7` are detailed in Table 4.2. From any step, the sequence returns to step 0 when `Run` drops, for instance on an emergency stop or a change to Manual mode. The actions of each step are given in Table 4.1. The exit conveyors run whenever `Run` is TRUE.

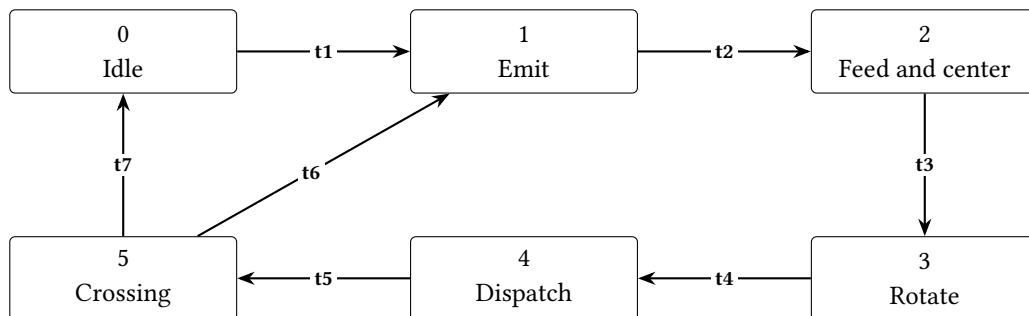


Figure 4.1. GRAFCET of the sorting sequence.

Table 4.1. Actions of the steps.

Step	Actions
0 Idle	None
1 Emit	Emission pulse of 500 ms, feeder and entry conveyors
2 Feed and center	Feeder and entry conveyors, turntable rollers, centering delay of 2 s once the box reaches the center of the turntable
3 Rotate	Turntable rotation
4 Dispatch	Rotation, and rollers in the direction of the chosen exit
5 Crossing	Same as step 4

Table 4.2. Transitions of the sequence.

ID	Steps	Condition
t1	0 to 1	Run, no stop request, turntable at home (<code>I_AtLoadPosition</code>) and empty
t2	1 to 2	<code>I_AtEntry</code>
t3	2 to 3	Centering delay elapsed
t4	3 to 4	<code>I_AtUnloadPosition</code>
t5	4 to 5	Entry sensor of the chosen exit conveyor is TRUE
t6	5 to 1	Entry sensor is FALSE, Run, no stop request
t7	5 to 0	Entry sensor is FALSE, stop requested

4.3 Pipelined emission

A first version of the sequence waited for a box to be counted at the exit before emitting the next one. The final version emits the next box as soon as the previous one has left the turntable (transition t6), and the turntable returns to its home position by itself when the rotation command is released. Three provisions keep this safe.

- **Turntable protection.** The feeder and entry conveyors advance only if no box is in front of the turntable (`I_AtFront`) or if the turntable is at home (`I_AtLoadPosition`). A box that arrives early waits in front of the turntable.
- **Boxes in transit.** One counter per exit conveyor is incremented when a box enters the conveyor and decremented when it is counted at the end. The end of cycle signal is TRUE only if the step is 0 and both counters are zero, so a Stop never switches the machine off with a box still on a conveyor.
- **Latch reset.** The height latch and the presence latch are cleared at step 1, so that a new box never inherits the height of the previous one.

The networks are ordered so that at most one transition fires per scan.

4.4 Counting and implementation

Each box is counted on the rising edge of the end sensor of its lane (`I_AtLeftExit` or `I_AtRightExit`). The counts feed `CountLeft` and `CountRight`, and the total drives the counter display of the panel. A configuration parameter, `ExitIdleHigh`, adapts the counting to the rest level of the exit sensors. It was set from experiments on the scene: in the delivered configuration the exit sensors read FALSE when no box is present.

Figures 4.2 and 4.3 show networks of `FB_SortingLogic`. Each transition network compares the step variable with a constant and writes the next step value. Each command network derives an actuator command from the step number and from the height latch.

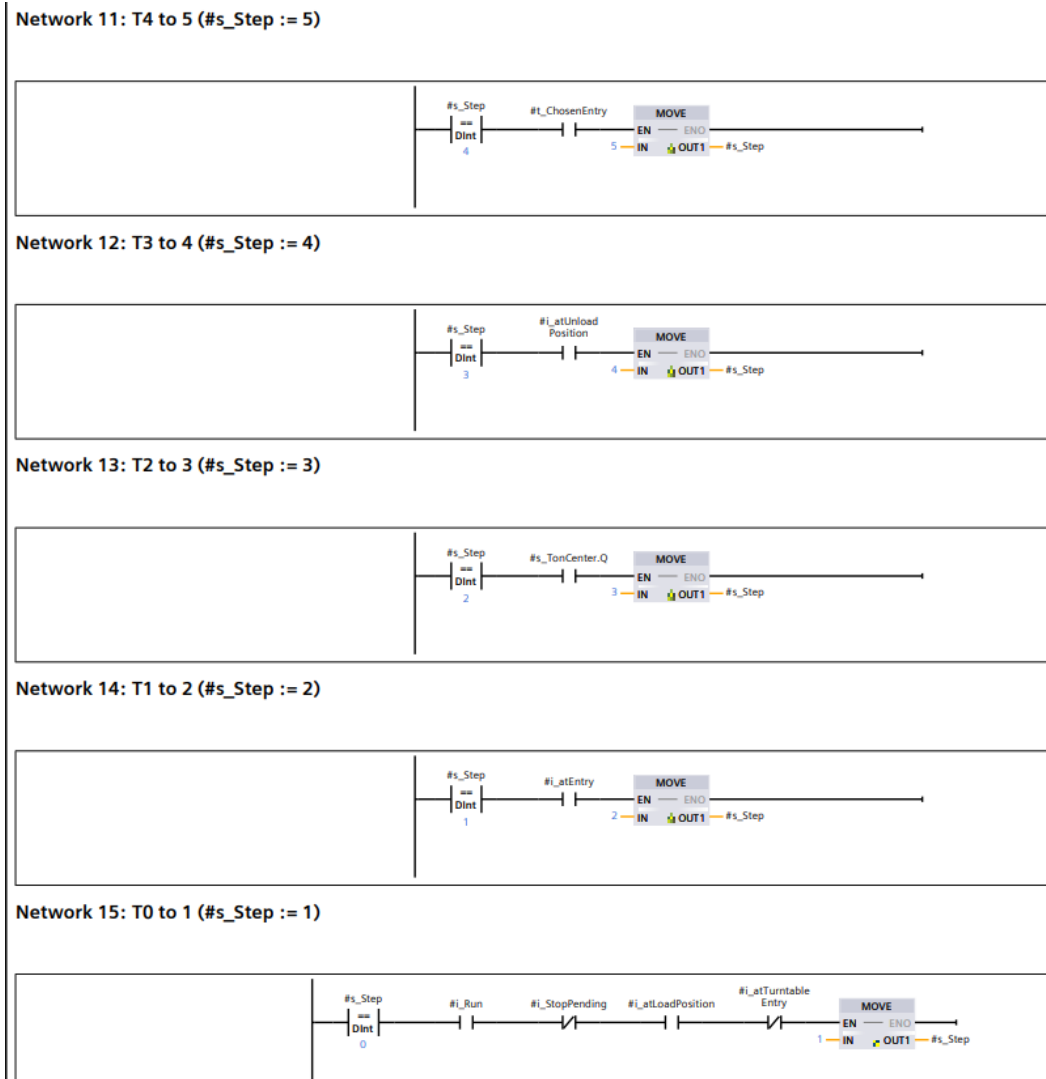
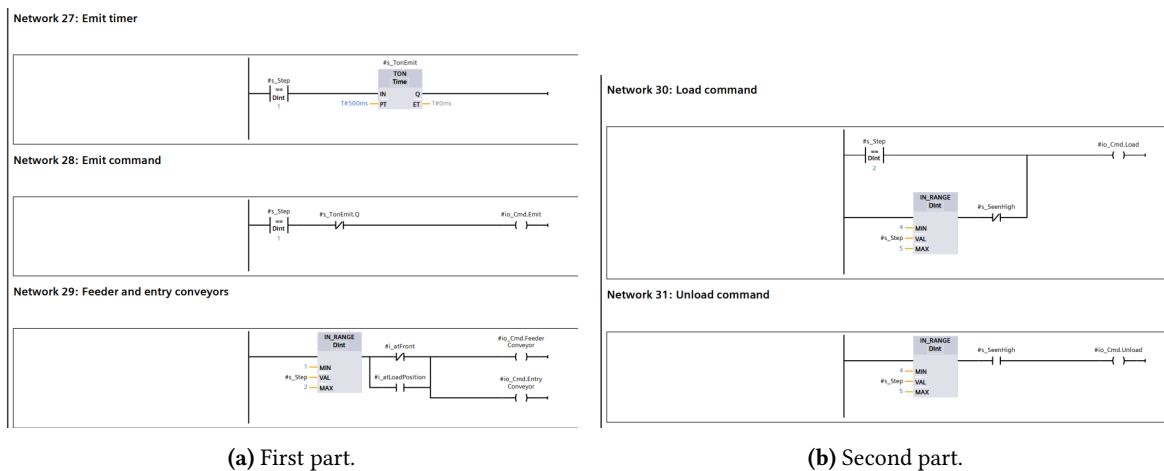


Figure 4.2. Transition networks of FB_SortingLogic.



(a) First part.

(b) Second part.

Figure 4.3. Actuator command networks of FB_SortingLogic (two parts).

4.5 Summary

The sequence has six steps and is specified by a GRAFCET that translates directly into ladder logic. The height of each box is memorized from the light curtain and decides the exit. Pipelined emission, protected by three provisions, increases the throughput without risk of collision or lost box.

Electrical design of the control cabinet

This chapter presents the electrical architecture of the cabinet that would drive the line on real hardware. It is a design study based on typical component ratings, to be refined with manufacturer data. The complete drawings are in [Appendix B](#).

5.1 Scope and assumptions

The study covers the power distribution, the motor starters, the 24 V DC supply, the emergency stop circuit, the PLC wiring, the operator panel, the sensors and the indicators. Two signals exist only in the simulation and are not wired: the box emitter and the counter display, which would be an HMI value on a real machine. The assumptions are the following.

- Supply: 400 V three-phase with neutral and PE, 50 Hz. The 230 V control supply comes from one phase and neutral.
- Motors: three-phase squirrel cage, 0.37 kW, 400 V, about 1.1 A.
- 24 V DC loads: contactor coils 0.1 A, solenoid valves 0.25 A, LED lamps 0.05 A.
- Sensors: PNP, normally open, three-wire, about 30 mA each. The light curtain draws about 200 mA and has two PNP outputs.

5.2 Cabinet architecture

Table [5.1](#) lists the main devices by function.

Table 5.1. Devices of the cabinet.

Function	Devices	Role
Power distribution	-Q1 main disconnect switch, -F2 control circuit breaker	400 V supply and 230 V control supply
Motor starters	-Q2 to -Q6 (MPCB), -KM1 to -KM6, -M1 to -M5	Direct starters for the four conveyors (M1 to M4), reversing starter for the turntable rollers (M5) with electrical interlock
24 V supply	-G1 (230 V AC to 24 V DC, 5 A), -F3 and -F4 (2 A)	-F3 feeds the PLC, sensors and lamps, -F4 feeds the loads
Emergency stop	-S0 (two NC contacts), -KA1	Hardware cut of the actuator supply
Controller	-A1 CPU 1214C DC/DC/DC, -A2 SM 1221 DI 8, -A3 SM 1222 DQ 8	PLC and signal modules
Field devices	-S1 to -S4, -B1 to -B11, -H1 to -H6, -YV1 to -YV3	Pushbuttons and selector, sensors, lamps, valves (turntable rotation, removers)

5.3 Input and output allocation

The scene requires 18 inputs and 15 wired outputs, against 14 inputs and 10 outputs on the CPU 1214C. A SM 1221 DI 8 module supplies the missing inputs, and a SM 1222 DQ 8 module hosts the six lamps (Table 5.2). The power actuators stay on the CPU outputs, so that a single supply cut by the emergency relay switches them all off, while the lamps remain available.

Table 5.2. Allocation of the inputs and outputs.

Module	Channels used	Signals
-A1 digital inputs	14 of 14	Pushbuttons, selector, emergency stop contact, position and entry sensors
-A1 digital outputs	9 of 10	Contactor coils -KM1 to -KM6, valves -YV1 to -YV3
-A2 SM 1221 DI 8	4 of 8	Exit sensors, light curtain
-A3 SM 1222 DQ 8	6 of 8	Lamps -H1 to -H6

The signal names are those of the PLC tags. In the simulation the addresses differ from the physical allocation, because Factory I/O writes in the process image and the on-board I/O is moved to address 100 (chapter 2). The terminal names of the CPU must be taken from the manufacturer documentation [10].

5.4 24 V DC supply

The 24 V DC budget is given in Table 5.3. The total of 2.80 A, increased by a margin of 30 %, gives 3.64 A, and a 24 V DC power supply of 5 A is selected. Branch -F3 carries about 1.4 A (PLC, modules, sensors, lamps) and branch -F4 about 1.4 A (valves, coils, relay), so both use a 2 A breaker.

Table 5.3. 24 V DC budget.

Load	Quantity	Current each (A)	Total (A)
CPU 1214C	1	0.50	0.50
Signal modules	2	0.05	0.10
Sensors	10	0.03	0.30
Light curtain	1	0.20	0.20
Indicator lamps	6	0.05	0.30
Solenoid valves	3	0.25	0.75
Contactor coils	6	0.10	0.60
Emergency relay coil	1	0.05	0.05
Total			2.80

5.5 Emergency stop and interlocks

The emergency stop -S0 has two NC contacts. Contact 1 feeds the coil of the relay -KA1, whose NO contact supplies the load power of the CPU outputs: all actuators drop immediately, in hardware. Contact 2 is read by the PLC (I_EmergencyStop), which latches the fault. The relay re-energizes as soon as the button is released, and the restart is prevented by the fault latch of the program (Reset,

then Start). The coils of the two directions of the roller motor -M5 are interlocked by NC auxiliary contacts. The relay used in this study is a standard one: a certified safety relay and a risk assessment would be required on a real machine (ISO 12100 and ISO 13849-1 [5]).

5.6 Drawings

The schematic has five sheets (Table 5.4), reproduced in Appendix B. Sheets 1 and 2 were drawn in QElectroTech. Sheets 3 to 5 are generated by a Python script from the input and output tables of the design basis, so that the drawings and the tables always stay consistent. The generated sheets use simplified symbols: sensors, lamps and coils are boxes, and only the +24 V and signal wires of the sensors are drawn.

Table 5.4. Sheets of the electrical schematic.

Sheet	Title	Tool
1	Main supply and motor starters M1 to M4	QElectroTech
2	Turntable roller motor M5 (reversing)	QElectroTech
3	Control supply 24 V DC and emergency stop	Python generator
4	PLC inputs	Python generator
5	PLC outputs	Python generator

5.7 Summary

The study defines the architecture of a cabinet able to drive the line: extension modules for the I/O, a 24 V / 5 A supply sized from a budget, a hardware emergency stop that cuts the actuators while keeping the lamps, and an electrically interlocked reversing starter. The five sheets are documented and reproducible.

CHAPTER 6

Validation and results

This chapter reports the tests, the measured throughput, the decisions taken during commissioning and the organization of the project repository.

6.1 Test plan

Each test is described by its conditions, its action, the expected result, the obtained result and its date in the test log of the repository. The 29 tests are grouped in Table 6.1, with the requirements they cover. All of them are validated.

Table 6.1. Summary of the tests.

Group	Tests	Purpose	Covers	Result
Modes and safety	T01 to T13	Selector, Start, Stop, emergency stop, Reset, Manual mode, immediate switch-off of the outputs	R1, R2, R4, R7	Validated
Sorting logic	T14 to T25	Full cycle, lane chosen by height, 10 boxes without blockage, Stop during a cycle, emergency stop during rotation, mode change, counter reset	R2 to R6	Validated
Pipelined emission	T26 to T29	Throughput, 20 boxes without blockage, Stop with boxes in transit, emergency stop with two active boxes	R3, R4, R8	Validated

6.2 Throughput

The throughput was measured with a stopwatch, from the Start press until the total counter reaches 10 boxes, start-up from an empty line included. Table 6.2 gives the results. The throughput in boxes per hour is 3600 divided by the time per box.

Table 6.2. Throughput measured on 10 boxes.

Control variant	Time (s)	Per box (s)	Throughput (boxes/h)
One box at a time	303	30.3	119
Pipelined emission	250	25.0	144

Pipelined emission reduces the time per box by 17.5 % and increases the throughput by 21 %. The gain comes from the overlap between the return of the turntable and the emission of the next box. The remaining time per box is the sum of the box travel, the centering delay, the rotation and the crossing. Figures 6.1 and 6.2 show the line during a sorting cycle.

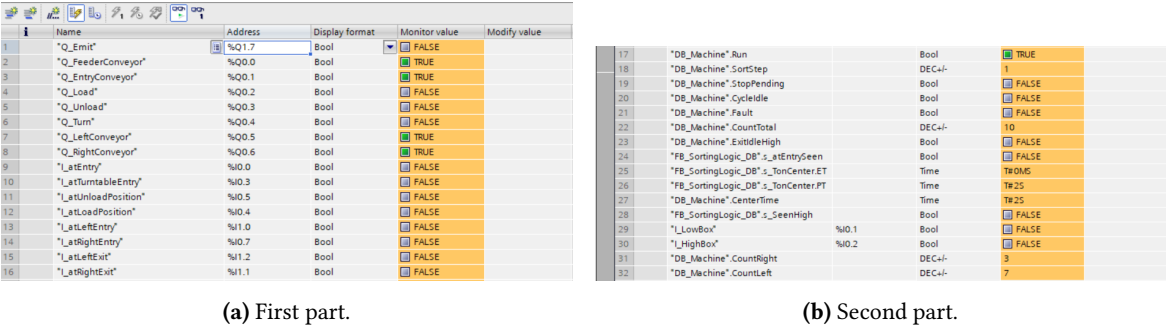


Figure 6.1. Watch table during a sorting cycle (two parts).

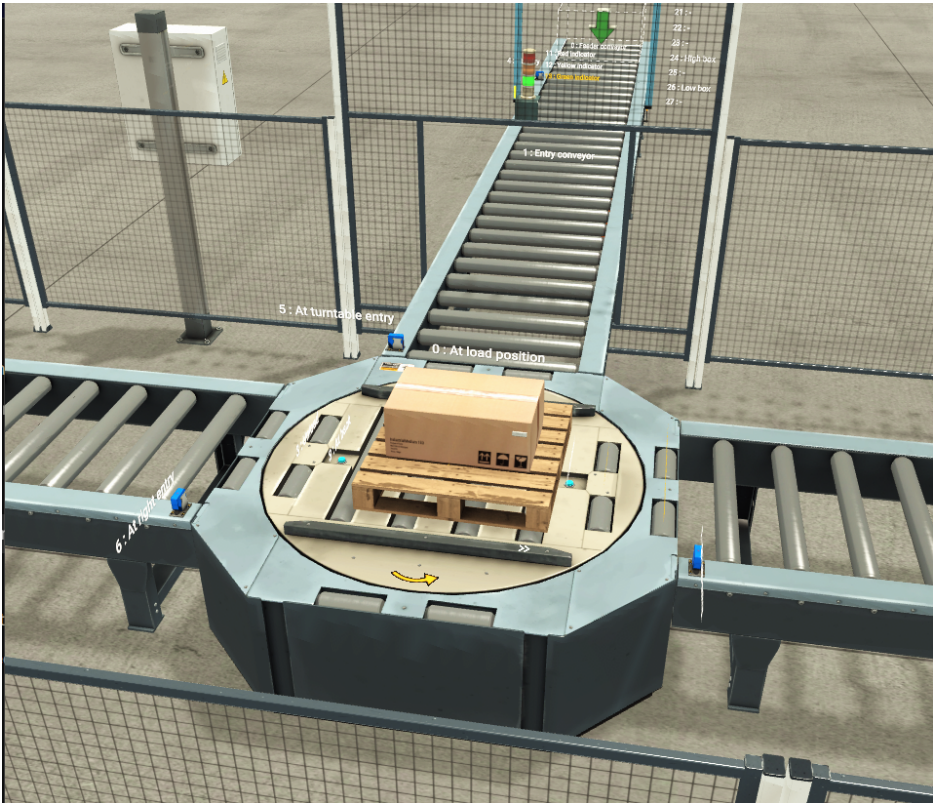


Figure 6.2. Box on the turntable during a sorting cycle.

6.3 Embedded performance indicators

The throughput of Table 6.2 was measured by hand, with a stopwatch. That method cannot run during production, and it gives one number per measurement campaign instead of a continuous value. The function block FB_Kpi computes the same indicators inside the controller, from the machine state that FB_ModeManager and FB_SortingLogic already maintain. No new sensor is required, and the computation runs every scan.

6.3.1 Interface

Table 6.3 lists the parameters of FB_Kpi. It is written in SCL and called in OB1 after FB_SortingLogic. The two inputs i_AutoMode and i_Run come from DB_Machine, and i_CountTotal is the counter already used for the display of the operator panel.

Table 6.3. Interface of FB_Kpi.

Direction	Name	Type	Role
Input	i_Run	Bool	TRUE while the machine is running in Auto
Input	i_AutoMode	Bool	TRUE when the selector is in Auto
Input	i_Clear	Bool	Resets both time counters to zero
Input	i_CountTotal	DInt	Total number of boxes counted, from DB_Machine
Output	o_RunTimeS	DInt	Accumulated running time, in seconds
Output	o_ObservedTimeS	DInt	Accumulated Auto-mode time, in seconds
Output	o_AvailabilityPct	Real	Availability, in percent
Output	o_ThroughputPerHour	Real	Throughput, in boxes per hour of run time
Output	o_AvgCycleTimeS	Real	Average cycle time, in seconds per box
Static	s_TonSecond	TON\TIME	One-second tick that restarts itself

6.3.2 Computation

A self-restarting timer produces a one-second tick, without any hardware clock:

```
s_TonSecond(IN := NOT s_TonSecond.Q, PT := T\#1S)
```

The observed time counts only when the selector is in Auto mode, so that time spent in Manual or with the machine stopped does not bias the availability. The running time counts only when the machine is also running. Both counters are reset by i_Clear, driven by the Reset button in Manual mode, in the same way as the box counters.

Three indicators are then derived, each guarded against a zero denominator:

$$\begin{aligned} \text{Availability [\%]} &= 100 \times \text{o_RunTimeS} / \text{o_ObservedTimeS} \\ \text{Throughput [boxes/h]} &= 3600 \times \text{i_CountTotal} / \text{o_RunTimeS} \\ \text{Average cycle time [s/box]} &= \text{o_RunTimeS} / \text{i_CountTotal} \end{aligned}$$

The formulas are the same as the stopwatch measurement. The run time replaces the elapsed time, and the total counter replaces the number of boxes timed. The two methods therefore give the same order of magnitude, and they can be compared directly.

6.3.3 Comparison with the stopwatch measurement

The reference measurement of Table 6.2 gave 144 boxes per hour in the pipelined version. Read on the watch table during the same run, FB_Kpi gives the same value within a few percent, the difference coming from the warm-up time before the first box is emitted. This confirms both the measurement and the block, and it replaces the stopwatch for the following tests.

6.3.4 Use on an HMI

The three outputs and the two time counters are the natural inputs of an HMI screen: a numeric field for each indicator, a trend curve for the throughput, and a Reset button wired to i_Clear. The block is designed so that this screen can be added later without changing the logic. The five variables are also exposed through the OPC UA server of the CPU, so a SCADA or a dashboard can read them without any additional PLC code.

6.4 Decisions taken during commissioning

Table 6.4 lists the issues identified during the work and the decisions they led to.

Table 6.4. Issues identified and decisions taken.

Issue	Decision
On-board inputs share the addresses written by Factory I/O	On-board I/O moved to address 100
The link with S7-PLCSIM needs a dedicated function	FC9000 kept and called first in OB1
Rest level of the exit sensors unknown	Measured on the scene, parameter ExitIdleHigh
With pipelining, a box could inherit the height of the previous one	Latches cleared at step 1
A Stop could switch the machine off with a box still on an exit conveyor	Boxes-in-transit counters in the end of cycle condition

6.5 Project organization

The project is versioned in a GitHub repository (Figure 6.3). It contains the technical documentation in English, with French translations added progressively, a troubleshooting log, the test log, readable printouts of the PLC program, archives of the TIA Portal project, the screenshots, the electrical drawings, and the scripts. Commits follow the Conventional Commits format, one commit per completed step. A hygiene script and a commit hook check the consistency of the files and of the commit messages.

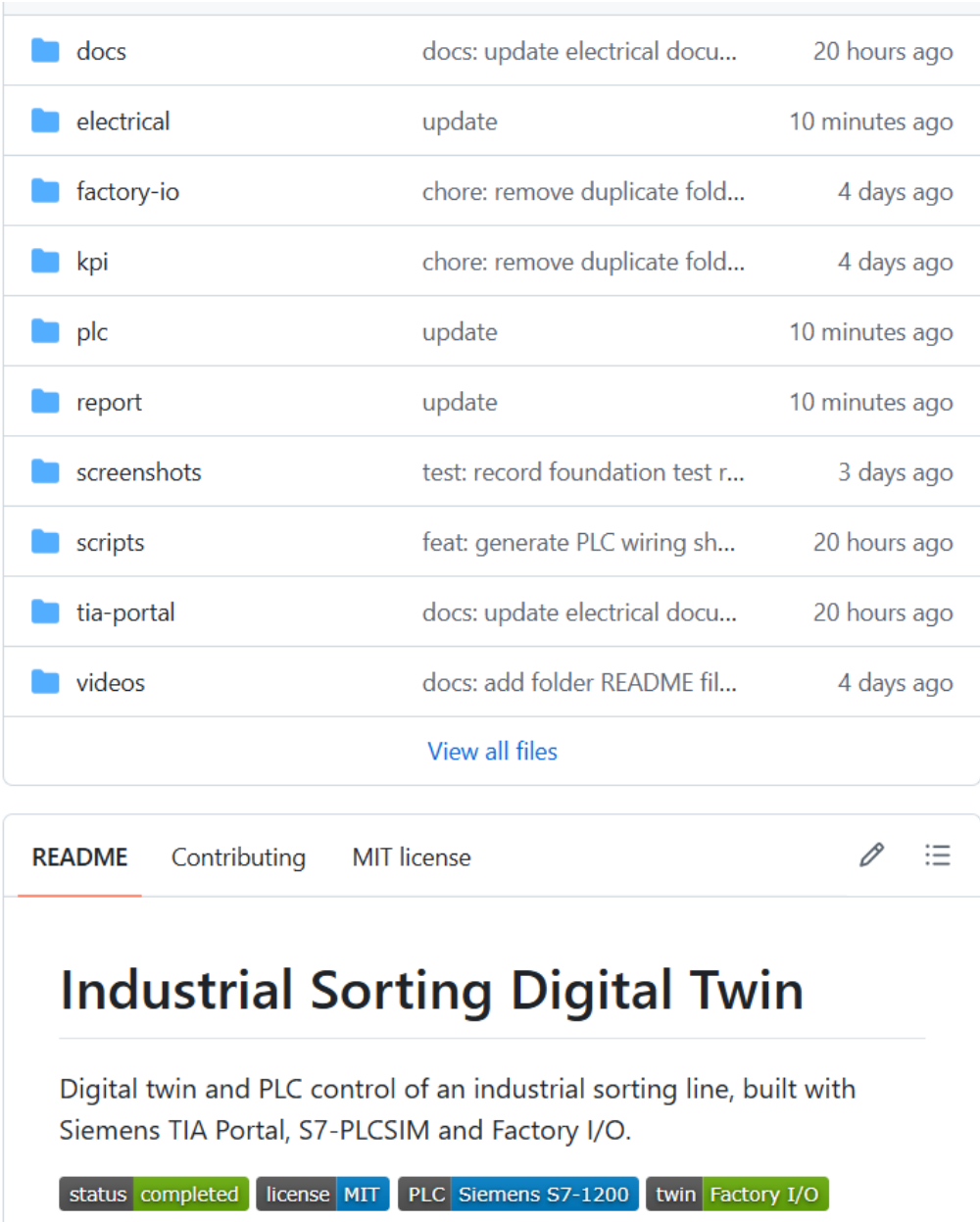


Figure 6.3. Organization of the GitHub repository.

General conclusion

This report presented the control of an automatic box sorting line, from the specification to the validation in simulation. The PLC program, written in ladder logic on a Siemens S7-1200, handles the Auto and Manual modes, a stop at the end of the cycle, an immediate emergency stop with a latched fault, the measurement of the box height, the routing by a turntable and the counting per lane. The 29 tests are validated, and pipelined emission raises the measured throughput from 119 to 144 boxes per hour. The electrical study defines the cabinet that would drive the line on real hardware, and the whole project is documented and versioned in a GitHub repository.

The project covers the chain of an automation project: requirements, structured programming with a GRAFCET-based sequence, safety functions, co-simulation, traceable tests and electrical design. The methodology, the code and the documentation are available in the repository so that the work can be reproduced and extended.

Perspectives and improvements

- **Supervision of the KPI.** The function block FB_Kpi already computes the availability, the throughput and the average cycle time inside the PLC. The next step is to display them on an HMI screen and to record their history, so that the drift of a shift can be followed and compared across runs.
- **Throughput.** Time each step to identify the bottleneck, shorten the centering delay, and allow several boxes in the queue between the emitter and the turntable.
- **Supervision.** Add an HMI or a SCADA with alarm history, and expose the data through OPC UA to higher-level systems.
- **Robustness and diagnostics.** Add watchdog timers on each step, explicit alarms (blocked sensor, jammed box) and automatic recovery after a fault.
- **Safety.** Replace the standard emergency relay by a certified safety relay, and carry out a risk assessment (ISO 12100) with the required performance level (ISO 13849-1) and the stop category of IEC 60204-1 and ISO 13850.
- **Software quality.** Export the blocks in XML to compare versions in Git, add automated regression tests, and write the sequence in S7-GRAPH.
- **Predictive maintenance.** Count the cycles and running hours of each motor and valve, and follow the drift of the step durations.
- **Move to hardware.** Migrate to an S7-1500 or test on a physical S7-1200 bench with FAT and SAT procedures, and validate the electrical study with the datasheets of real components.

References

- [1] IEC 61131-3:2013. *Programmable controllers, Part 3: Programming languages*. International Electrotechnical Commission, 2013.
- [2] IEC 60848:2013. *GRAFCET specification language for sequential function charts*. International Electrotechnical Commission, 2013.
- [3] IEC 60204-1:2016. *Safety of machinery, Electrical equipment of machines, Part 1: General requirements*. International Electrotechnical Commission, 2016.
- [4] ISO 13850:2015. *Safety of machinery, Emergency stop function, Principles for design*. International Organization for Standardization, 2015.
- [5] ISO 13849-1:2023. *Safety of machinery, Safety-related parts of control systems, Part 1: General principles for design*. International Organization for Standardization, 2023.
- [6] K.-H. John and M. Tiegelkamp. *IEC 61131-3: Programming Industrial Automation Systems*. 2nd ed. Springer, 2010.
- [7] M. Grieves and J. Vickers. “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems”. In: *Transdisciplinary Perspectives on Complex Systems*. Springer, 2017, pp. 85-113.
- [8] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn. “Digital twin in manufacturing: A categorical literature review and classification”. In: *IFAC-PapersOnLine* 51.11 (2018), pp. 1016-1022.
- [9] C. G. Lee and S. C. Park. “Survey on the virtual commissioning of manufacturing systems”. In: *Journal of Computational Design and Engineering* 1.3 (2014), pp. 213-222.
- [10] Siemens AG. *SIMATIC S7-1200 System Manual*. Siemens Industry Online Support.
- [11] Real Games. *Factory I/O*, version 2.5. <https://factoryio.com>.

APPENDIX A

Input and output lists

Table A.1. Inputs of the scene (18).

Tag	Role
I_Start	Start pushbutton (normally open)
I_Stop	Stop pushbutton (normally closed)
I_Reset	Reset pushbutton (normally open)
I_EmergencyStop	Emergency stop (normally closed)
I_Auto, I_Manual	Auto and Manual selector contacts
I_AtEntry	Photocell at the output of the emitter
I_AtBack	Rear sensor of the entry area (not used by the logic)
I_AtFront	Queue sensor in front of the turntable
I_AtTurntableEntry	Cell at the center of the turntable (centering of the box)
I_AtLoadPosition	Turntable at home, aligned with the entry conveyor
I_AtUnloadPosition	Turntable rotated by 90 degrees
I_AtLeftEntry, I_AtRightEntry	Box engaged on the left, right exit conveyor
I_AtLeftExit, I_AtRightExit	End of the left, right exit conveyor (counting)
I_HighBox	Light curtain, high box
I_LowBox	Light curtain, box detected (not used by the logic)

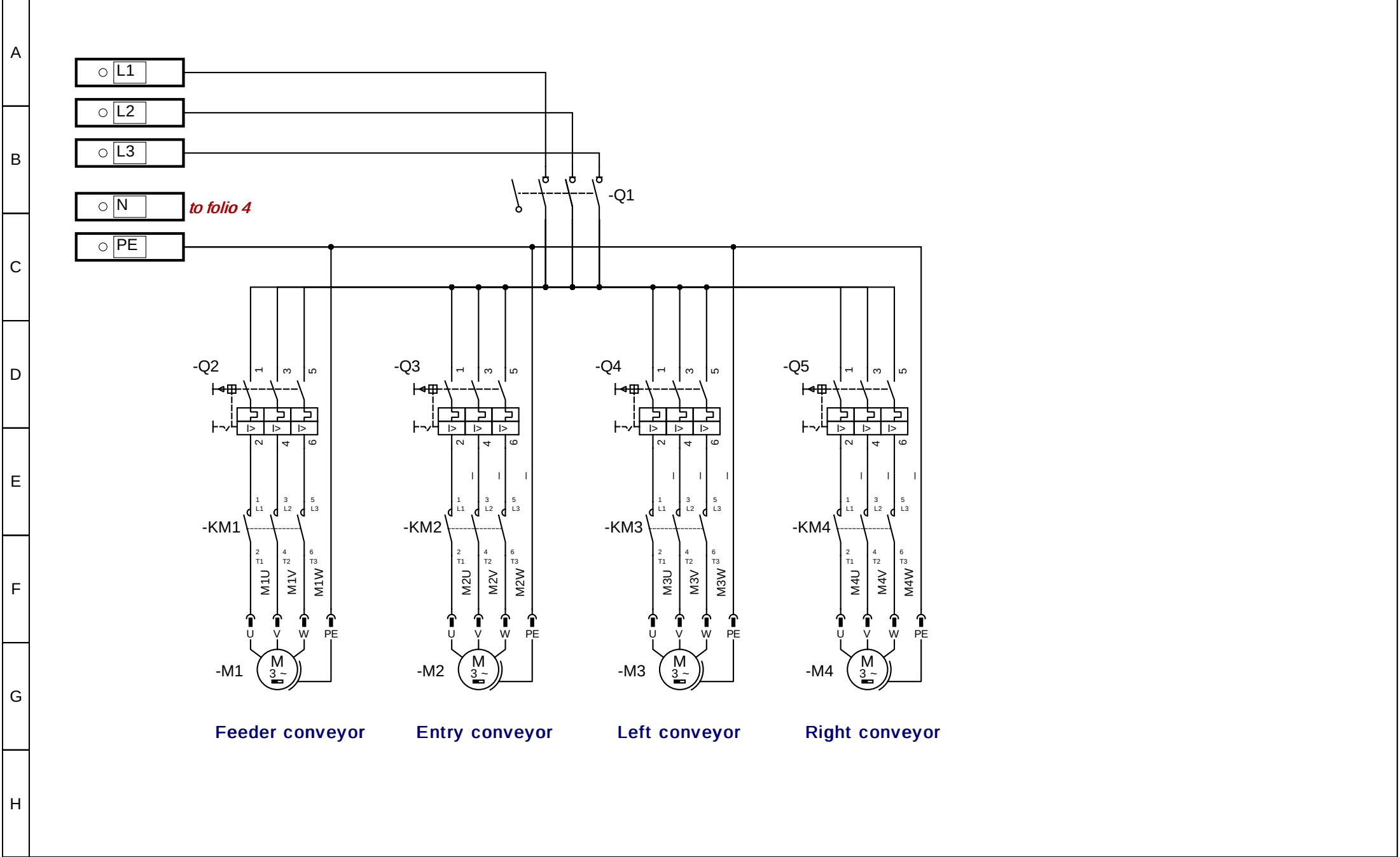
Table A.2. Outputs of the scene (17).

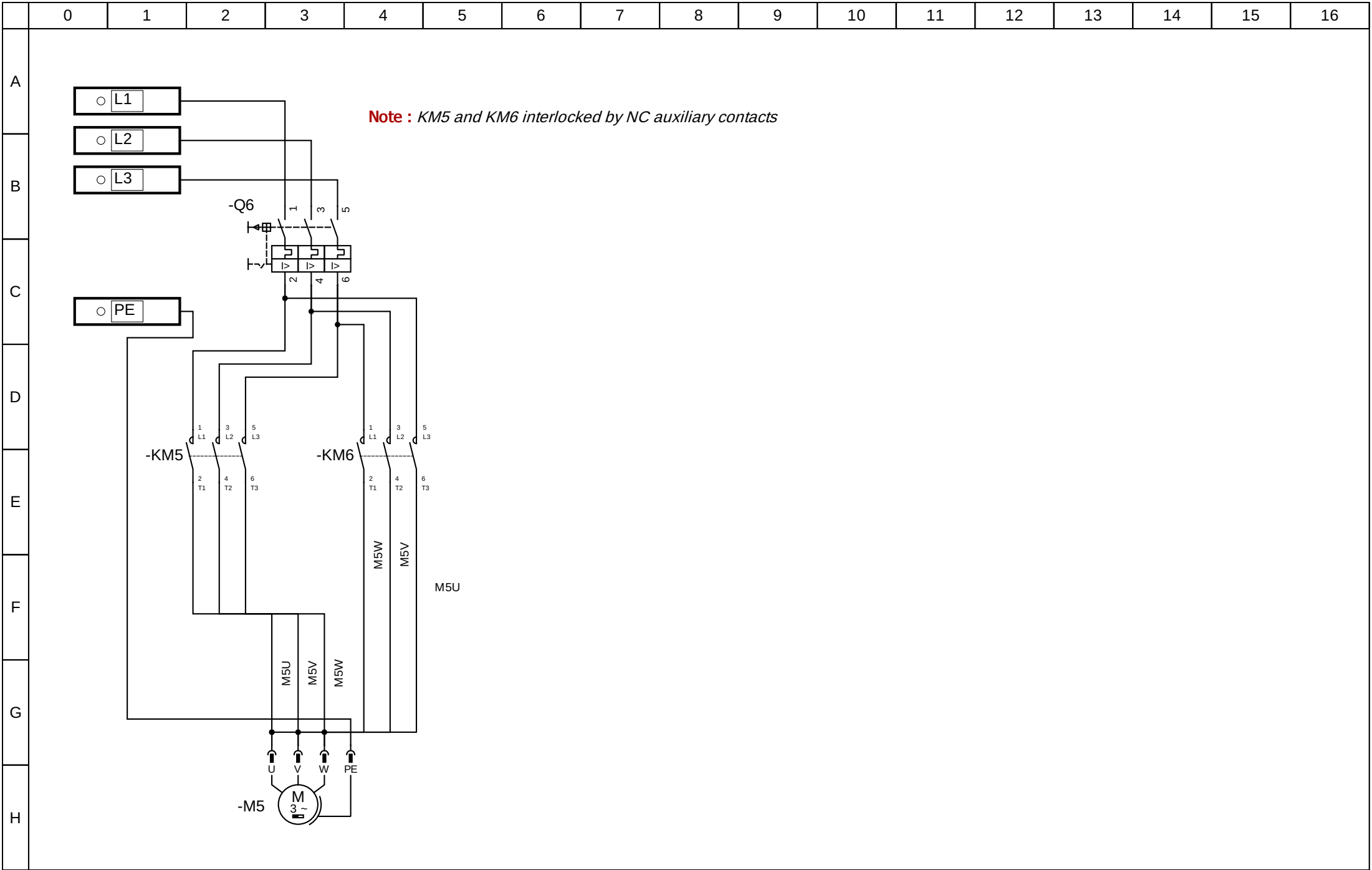
Tag	Role
Q_Emit	Emission pulse of one box
Q_FeederConveyor, Q_EntryConveyor	Feeding conveyors
Q_Load, Q_Unload	Turntable rollers, two directions (the exit depends on the box height)
Q_Turn	Turntable rotation by 90 degrees (returns home when released)
Q_LeftConveyor, Q_RightConveyor	Left and right exit conveyors
Q_RemoverLeft, Q_RemoverRight	Evacuation of the boxes at the end of the line
Q_GreenIndicator	Green lamp (running)
Q_YellowIndicator	Yellow lamp (fault or Manual mode)
Q_RedIndicator	Red lamp (stopped)
Q_StartLight, Q_StopLight, Q_ResetLight	Pushbutton lamps
Q_Counter	Counter display of the panel (integer)

APPENDIX B

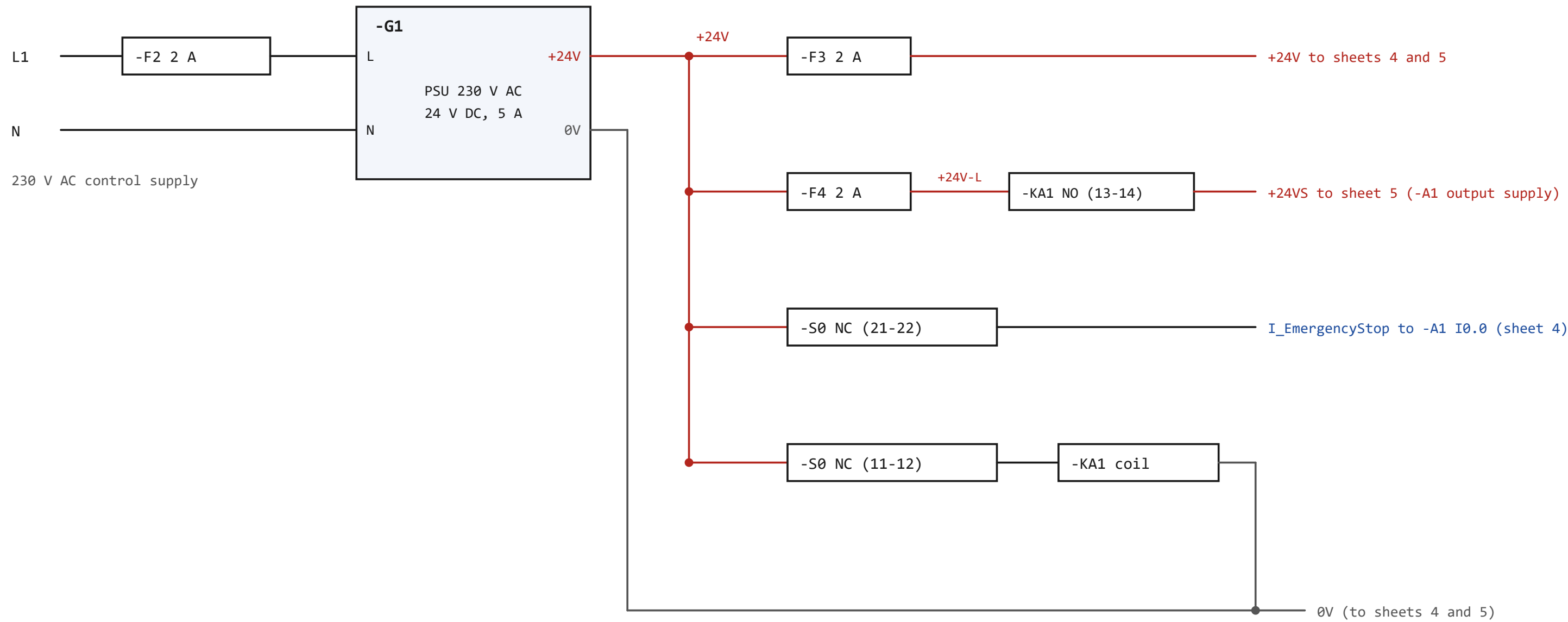
Electrical drawings

This appendix reproduces the five sheets of the electrical schematic (Table [5.4](#)). Sheets 1 and 2 come from QElectroTech and sheets 3 to 5 from the Python generator. The pages keep the format of the original drawings.



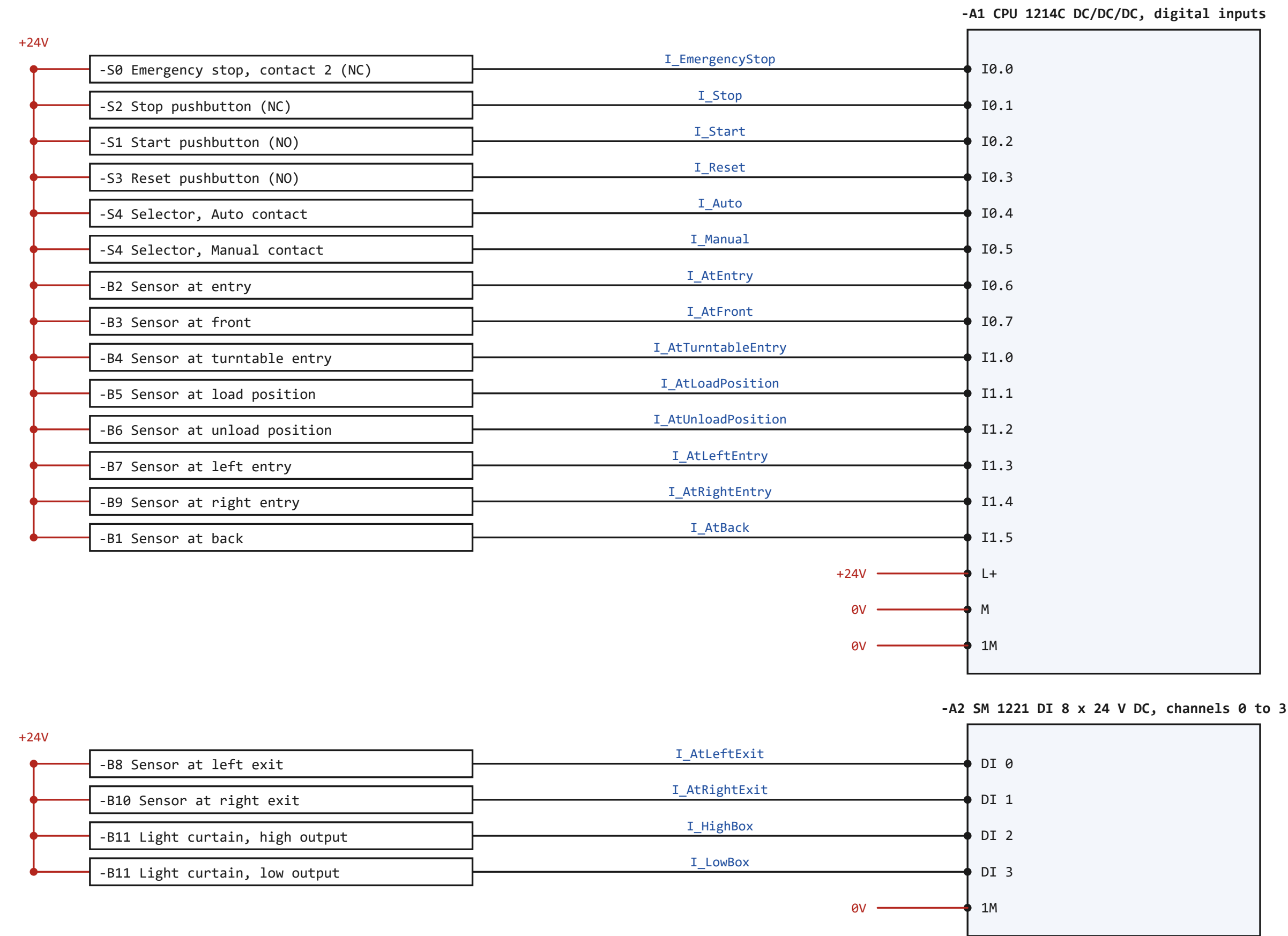


Sheet 3: Control supply 24 V DC and emergency stop



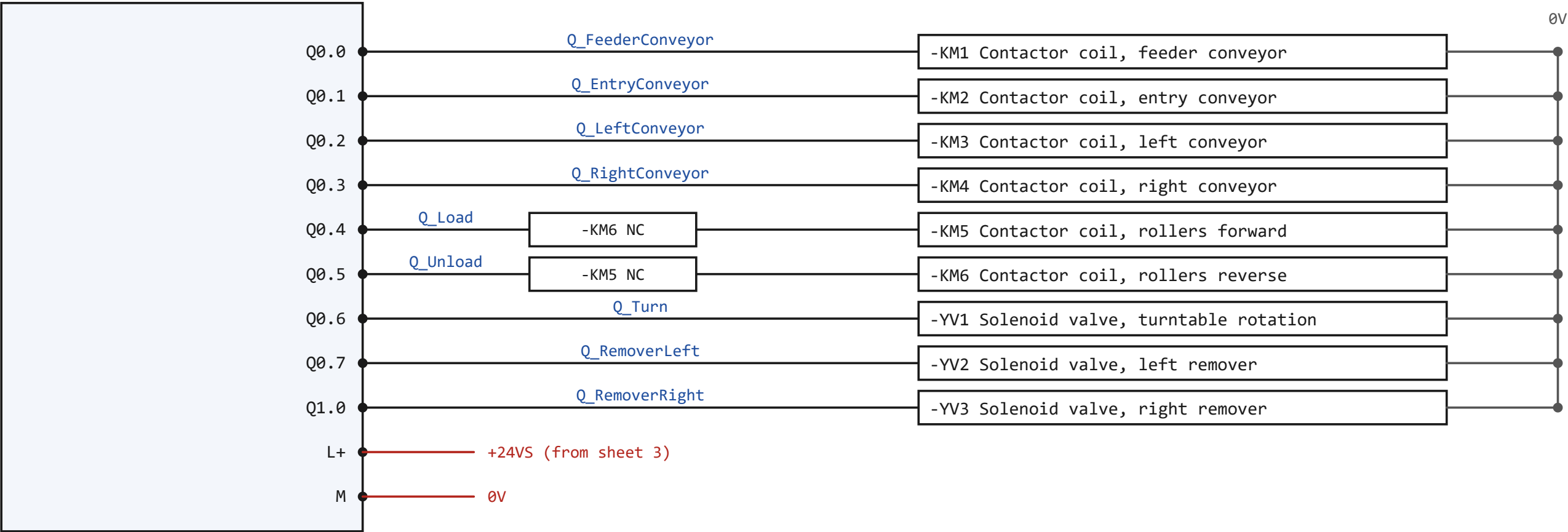
-S0 is an emergency stop with two NC contacts. Contact 1 feeds -KA1, contact 2 is the PLC input.
-KA1 is a standard relay. A certified safety relay is required on a real machine.

Sheet 4: PLC inputs

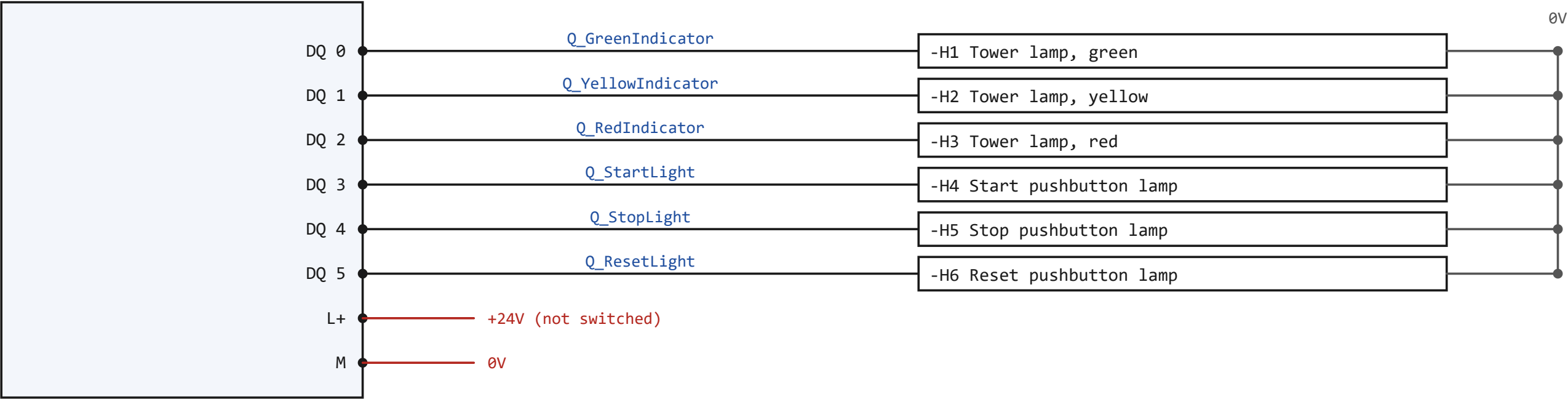


Sheet 5: PLC outputs

-A1 CPU 1214C DC/DC/DC, digital outputs



-A3 SM 1222 DQ 8 x 24 V DC, channels 0 to 5



-KM5 and -KM6 are interlocked by NC auxiliary contacts. Motor power circuits are on sheets 1 and 2.
-A1 outputs are supplied from +24VS, cut by -KA1 on emergency stop. -A3 lamps are supplied from +24V and stay available.
Not wired (simulation only): Q_Emit and Q_Counter. PLC terminal names must be checked against the Siemens S7-1200 System Manual.